

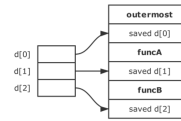
2019

Problem 2: Static Link and Escape Calculation (30 points)

```

1  /* A program to solve the 1-N queens problem */
2
3  let
4  var maxN := 10
5  var emptySymbol := " ."
6
7  function printSquare(hasQueen:int) =
8  let
9  var queenSymbol := " Q"
10 in print(if hasQueen then queenSymbol else emptySymbol) end
11
12 function nQueens(N:int) =
13 let
14 type intArray = array of int
15
16 var row := intArray [ N ] of 0
17 var col := intArray [ N ] of 0
18 var diag1 := intArray [N+N-1] of 0
19 var diag2 := intArray [N+N-1] of 0
20
21 function printBoard() =
22 (for i := 0 to N-1
23 do for j := 0 to N-1
24 do printSquare(col[i]=j); print("\n");
25 print("\n"))
26
27 function try(c:int) = (
28 if c = N then printBoard()
29 else for r := 0 to N-1
30 do if row[r] = 0 & diag1[r+c] = 0 & diag2[r-c+N-1] = 0
31 then (row[r] := 1; diag1[r+c] := 1; diag2[r-c+N-1] := 1;
32 col[c] := r;
33 try(c+1);
34 row[r] := 0; diag1[r+c] := 0; diag2[r-c+N-1] := 0)
35 )
36 in try(0) end
37 in
38 for n := 1 to maxN
39 do (print("Solving "); print(n); print(" queens problem. . . \n\n");
40 nQueens(n);
41 print("\n"))
42 end

```

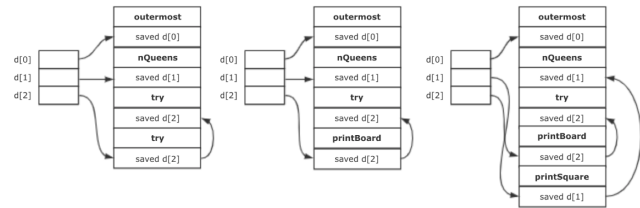


3. Determine whether the following local variables can escape and explain your reasons. (12')

Variable	Escape(Y/N)	Your Reason
maxN (Line 4)		
emptySymbol (Line 5)		
queenSymbol (Line 9)		
col (Line 17)		

Variable	Escape(Y/N)	Your Reason
maxN (Line 4)	N	没有被嵌套的函数访问，而且是简单的数据类型，能放入寄存器中
emptySymbol (Line 5)	Y	字符串变量是指向字符串的指针，虽然可以存放寄存器中，但被 printSquare 函数访问
queenSymbol (Line 9)	N	字符串变量是指向字符串的指针，只被当前函数所访问
col (Line 17)	Y	col 被嵌套的 try 函数访问，虽然 col 是一个数组类型，但是还是其数据是放在堆上的，col 也只是个指针

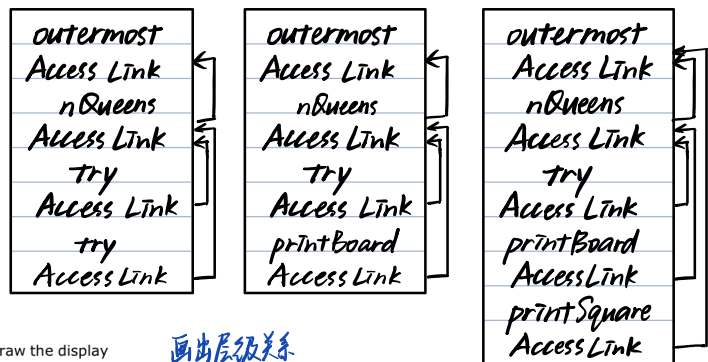
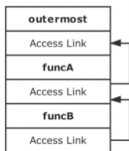
2.



指向最近一次进入的，其静态嵌套深度是 i 的过程的栈帧

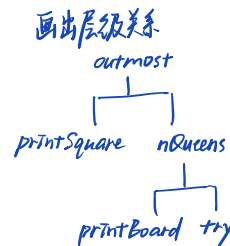
1. Suppose that we implement the nested functions using an access link. Please draw the access link in each of the activation records on the stack based on the following function invocations. (9')

- (1) nQueens->try->try
 - (2) nQueens->try->printBoard
 - (3) nQueens->try->printBoard->printSquare
- e.g. funcA->funcB



2. Suppose that we implement nested functions using displays. Please draw the display in each of the activation records on the stack based on the following function invocations. (9')

- (1) nQueens->try->try
 - (2) nQueens->try->printBoard
 - (3) nQueens->try->printBoard->printSquare
- e.g. funcA -> funcB



2018

Problem 2: Static Link (25 points)

```

let
  type IntArray = array of int
  var len := 16
  var arr := IntArray [len] of 0

  function swap(a:int, b:int) =
    let
      var t := arr[a]
    in
      arr[a] := arr[b]
      arr[b] := t
    end

  function sortArray(len:int) =
    let
      var head := 0
      var tail := len - 1
      function quickSort(low:int, high:int) =
        let
          var pi := 0
          function partition(low:int, high:int) : int =
            let
              var pivot := arr[high]
              var i := low - 1
            in
              for j := low to (high - 1)
              do if arr[j] <= pivot then
                  (i := i + 1; swap(i, j))
                  swap(i + 1, high); i + 1
              end
            in
              if low < high then
                (pi := partition(low, high);
                 quickSort(low, pi - 1);
                 quickSort(pi, high))
              end
            in
              quicksort(head, tail)
          end
        in
          sortArray(len)
        end
    end
  end

```

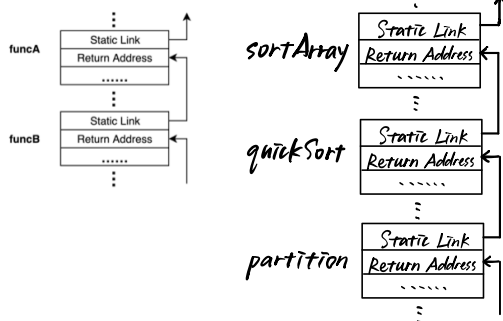
Handwritten annotations on the code:

- Blue arrows pointing from `swap` to `swap` and `partition` to `partition` within the `quickSort` function.
- Blue arrows pointing from `sortArray` to `quickSort` and `partition` to `partition` within the `sortArray` function.

1. Suppose that we implement the nested functions using an access link. Please draw the access link in each of the activation records on the stack based on the following function invocations (10')

- 1) sortArray → quickSort → partition
- 2) sortArray → quickSort → partition → swap
- 3) sortArray → quickSort → partition → swap → quickSort → partition

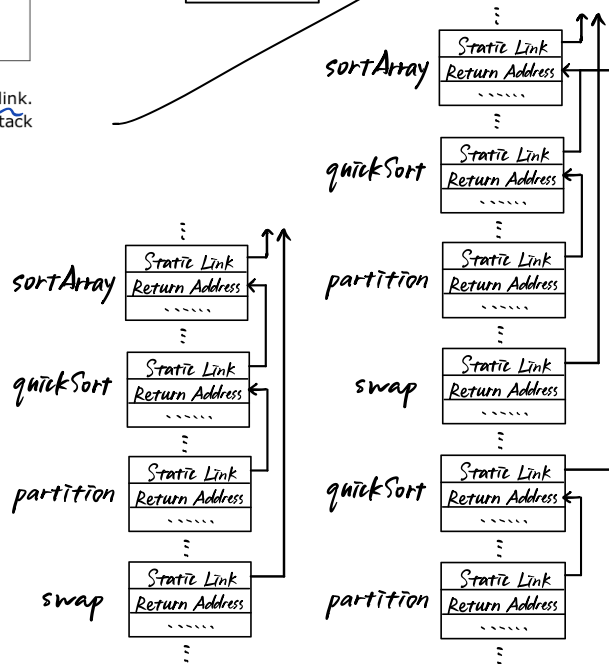
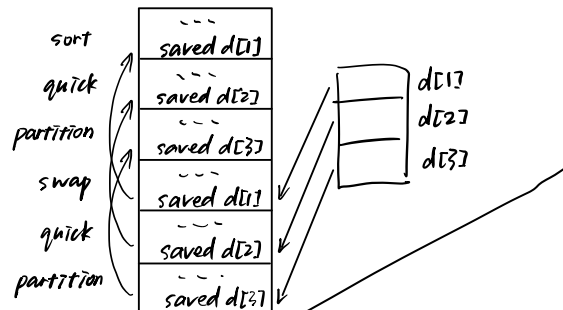
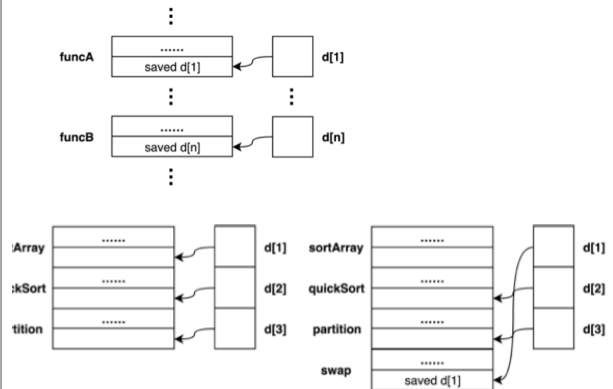
e.g.



2. Suppose that we implement nested functions using displays. Please draw the display in each of the activation records on the stack based on the following function invocations. (15')

- 1) sortArray → quickSort → partition
- 2) sortArray → quickSort → partition → swap
- 3) sortArray → quickSort → partition → swap → quickSort → partition

e.g.

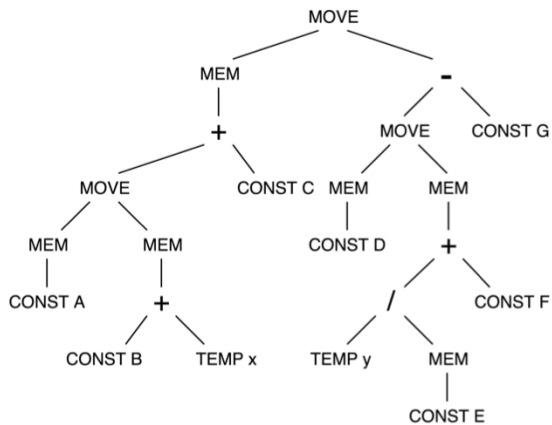


2018

Name	Effect	Trees	Cost
-	r_i	TEMP	0
ADD	$r_i \leftarrow r_j + r_k$	$ \begin{array}{c} + \\ / \quad \backslash \\ r_j \quad r_k \end{array} $	1
MUL	$r_i \leftarrow r_j * r_k$	$ \begin{array}{c} * \\ / \quad \backslash \\ r_j \quad r_k \end{array} $	1
SUB	$r_i \leftarrow r_j - r_k$	$ \begin{array}{c} - \\ / \quad \backslash \\ r_j \quad r_k \end{array} $	1
DIV	$r_i \leftarrow r_j / r_k$	$ \begin{array}{c} / \\ / \quad \backslash \\ r_j \quad r_k \end{array} $	1
ADDI	$r_i \leftarrow r_j + c$	$ \begin{array}{c} + \quad \text{CONST} \\ / \quad \backslash \\ \text{CONST} \quad \text{CONST} \end{array} $	1
SUBI	$r_i \leftarrow r_j - c$	$ \begin{array}{c} - \\ / \quad \backslash \\ \text{CONST} \quad \text{CONST} \end{array} $	1
LOAD	$r_i \leftarrow M[r_j + c]$	$ \begin{array}{cccc} \text{MEM} & \text{MEM} & \text{MEM} & \text{MEM} \\ & & & \\ + & & + & \text{CONST} \\ / \quad \backslash & & / \quad \backslash & \\ \text{CONST} & & \text{CONST} & \end{array} $	2
STORE	$M[r_j + c] \leftarrow r_i$	$ \begin{array}{cccc} \text{MOVE} & \text{MOVE} & \text{MOVE} & \text{MOVE} \\ / \quad \backslash & / \quad \backslash & / \quad \backslash & / \quad \backslash \\ \text{MEM} & \text{MEM} & \text{MEM} & \text{MEM} \\ & & & \\ + & + & \text{CONST} & \\ / \quad \backslash & / \quad \backslash & & \\ \text{CONST} & \text{CONST} & & \end{array} $	2
MOVEM	$M[r_j] \leftarrow M[r_i]$ $r_k \leftarrow M[r_i]$	$ \begin{array}{c} \text{MOVE} \\ / \quad \backslash \\ \text{MEM} \quad \text{MEM} \\ \quad \end{array} $	4

Note:

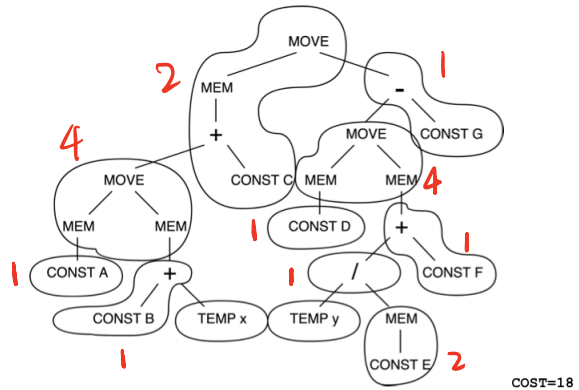
- The notation **$M[x]$** denotes the memory word at address x .
 - The **third column** in the table above is the set of **tiles** corresponding to the machine instruction in the first column of the table.
 - The **fourth column** in the table above is the **cost** corresponding to the machine instruction in the first column of the table.
- Consider the following IR tree and answer the questions below.



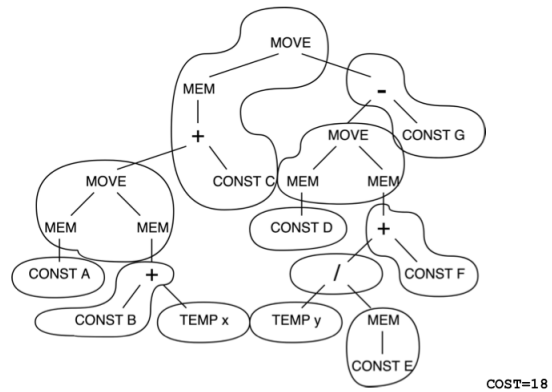
1. In the course, we have learnt **maximum munch algorithm** to tile the IR tree. Please use this algorithm to tile the IR tree above. **Draw your tiles, write the corresponding instruction sequences** after tiling, and **calculate the cost** of your instruction sequences. (10')
2. **Maximum munch algorithm** is an **optimal** tiling algorithm, and will not always generate tiles with smallest cost. A **dynamic-programming algorithm** can find the **optimum** tiling to the IR tree. Please use the dynamic-programming algorithm to tile the IR tree above. **Draw your tiles, write the corresponding instruction sequences** after tiling, and **calculate the cost** of your instruction sequences. (15')

Problem 3: (25 points)

1. (10', 4 分 munch 正确性, 4 分指令, 2 分计算 COST)



- 2 (15', 6 分 munch 正确性, 6 分指令, 3 分计算 COST)



2017

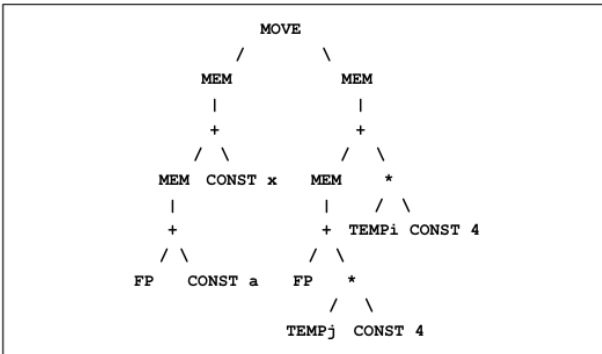
Problem 3: Instruction Selection (26 points)

Name	Effect	Trees	Cost
-	r_i	TEMP	1
ADD	$r_i \leftarrow r_j + r_k$	$\begin{array}{cc} + & * \\ / \backslash & / \backslash \end{array}$	1
MUL	$r_i \leftarrow r_j * r_k$	$\begin{array}{cc} - & / \\ / \backslash & / \backslash \end{array}$	1
SUB	$r_i \leftarrow r_j - r_k$	$\begin{array}{cc} + & + \\ / \backslash & / \backslash \end{array}$	1
DIV	$r_i \leftarrow r_j / r_k$	$\begin{array}{cc} + & + \\ / \backslash & / \backslash \end{array}$	1
ADDI	$r_i \leftarrow r_j + c$	$\begin{array}{cc} + & + \\ / \backslash & / \backslash \end{array}$ CONST	1
SUBI	$r_i \leftarrow r_j - c$	$\begin{array}{cc} - & \\ / \backslash & \\ & \text{CONST} \end{array}$	1
LOAD	$r_i \leftarrow M[r_j + c]$	$\begin{array}{cccc} \text{MEM} & \text{MEM} & \text{MEM} & \text{MEM} \\ & & & \\ + & & + & \text{CONST} \\ / \backslash & & / \backslash & \\ \text{CONST} & \text{CONST} & & \end{array}$	1
STORE	$M[r_j + c] \leftarrow r_i$	$\begin{array}{cccc} \text{MOVE} & \text{MOVE} & \text{MOVE} & \text{MOVE} \\ / \backslash & / \backslash & / \backslash & / \backslash \\ \text{MEM} & \text{MEM} & \text{MEM} & \text{MEM} \\ & & & \\ + & + & \text{CONST} & \\ / \backslash & / \backslash & & \\ \text{CONST} & \text{CONST} & & \end{array}$	1
MOVEM	$M[r_j] \leftarrow M[r_i]$	$\begin{array}{cc} \text{MOVE} & \\ / \backslash & \\ \text{MEM} & \text{MEM} \end{array}$	2

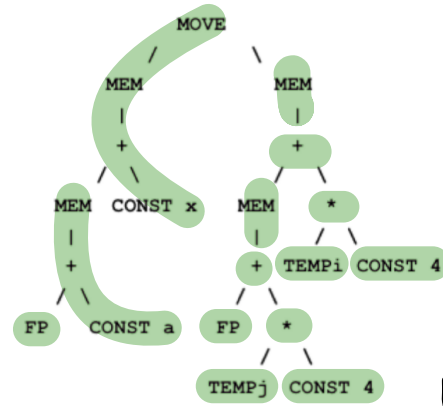
Note:

- The notation $M[x]$ denotes the memory word at address x .
- The **third column** in the table above is the set of **tiles** corresponding to the machine instruction in the first column of the table.
- The **fourth column** in the table above is the **cost** corresponding to the machine instruction in the first column of the table.

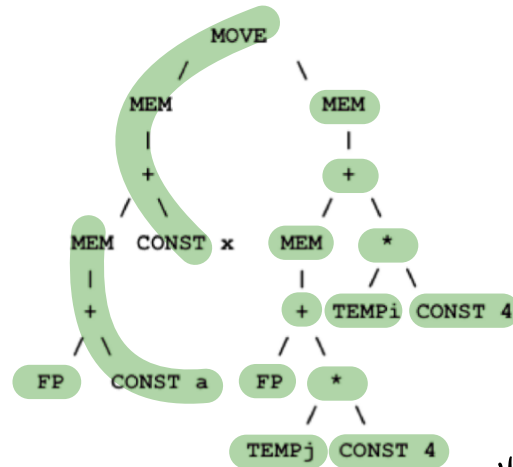
Consider the following IR tree and answer the questions below.



1. In the course, we have learnt **maximum munch algorithm** to tile the IR tree. Please use this algorithm to tile the IR tree above. **Draw your tiles, write the corresponding instruction sequences** after tiling, and **calculate the cost** of your instruction sequences. (13')
2. **Maximum munch algorithm** is an **optimal** tiling algorithm, and will not always generate tiles with smallest cost. A **dynamic-programming algorithm** can find the **optimum** tiling to the IR tree. Please use the dynamic-programming algorithm to tile the IR tree above. **Draw your tiles, write the corresponding instruction sequences** after tiling, and **calculate the cost** of your instruction sequences. (13')



14



14

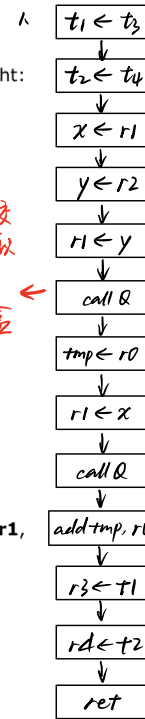
2019

Problem 4: Register allocation (34 points)

Suppose a compiler has compiled the following function **P** into instructions on the right:

long P(long x, long y)	P:
{	t1 <- r3
long u = Q(y);	t2 <- r4
long v = Q(x);	x <- r1
return u + v;	y <- r2
}	r1 <- y
	call Q
	tmp <- r0
	r1 <- x
	call Q
	add tmp, r0
	r3 <- t1
	r4 <- t2
	ret

call 比较特殊, 可以理解为一个小黑盒



2. call 指令:

def 为 caller-saved
use 为 callee-saved 寄存器

ret 指令:

def 为空, use 为空
tmp, out 为 callee-saved
和存函数返回值的寄存器

Several things are worth noting in the instructions:

- There are **five** hardware registers in all.
- The compiler passes parameters using registers. (The first parameter goes to **r1**, the second goes to **r2**)
- **r3, r4** is **callee-saved** while **r1, r2** are **caller-saved**.
- The **return value** will be stored in **r0**.
- **t1, t2, x, y, tmp** are temporary registers.

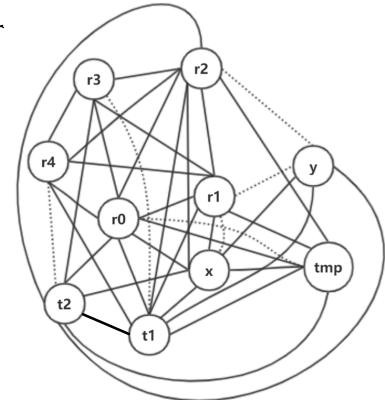
Please answer the following questions according to the instructions.

1. Draw a control flow graph instruction-by-instruction. (4')

2. Fill up the following def/use/in/out chart. (10')

instr	def	use	in	out
t1 <- r3	t1	r3	r1, r2, r3, r4	r1, r2, t4, t1
t2 <- r4	t2	r4	r1, r2, r4, t1	r1, r2, t1, t2
x <- r1	x	r1	r1, r2, t1, t2	r2, t1, t2, x
y <- r2	y	r2	r2, t1, t2, x	t1, t2, x, y
r1 <- y	r1	y	t1, t2, x, y	r1, x, t1, t2
call Q	r0, r1, r2	r1	r1, x, t1, t2	r0, x, t1, t2
tmp <- r0	tmp	r0	r0, x, t1, t2	tmp, x, t1, t2
r1 <- x	r1	x	tmp, x, t1, t2	tmp, r1, t1, t2
call Q	r0, r1, r2	r1	tmp, r1, t1, t2	tmp, r0, t1, t2
add tmp, r0	r0	tmp, r0	tmp, r0, t1, t2	r0, t1, t2
r3 <- t1	r3	t1	r0, t1, t2	r0, r3, t2
r4 <- t2	r4	t2	r0, r3, t2	r0, r3, r4
ret			r0, r3, r4	r0, r3, r4

3.



3. Draw the interference graph for the program. Please use dashed lines for move edges and solid line for real interference edges. (4')

4. Adopt the graph-coloring algorithm to allocate registers for temporaries. Write down the instructions after register allocation. (16')

NOTE: You must write down **every decision you make** such as simplifying, spilling, coalescing. And you also need to write down **new instructions** and draw **new interference graph after each spilling**. If your answer is too simple, you will get a part of score.

① 所有机器寄存器之间都是冲突的

② 冲突图是由 def 和出口活跃的 out 来产生冲突的 P62

4.

合并 y, r2 (George)

冻结 r0,r1,r3,r4

溢出 t1

溢出 t2

简化 x

简化 tmp

弹出栈分配颜色, t1,t2 成为实际溢出

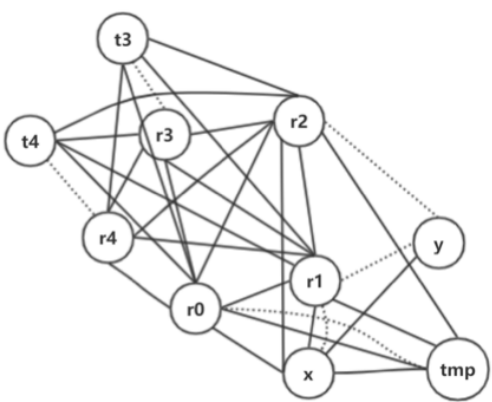
(下面步骤不生成 t3, t4 两个新的临时变量, 直接将 t1,t2 替换成内存也算正确)

分配栈空间, 重写指令:

```
t3 <- r3
M[t1] <- t3
t4 <- r4
M[t2] <- t4
x <- r1
y <- r2
r1 <- y
call Q
tmp <- r0
r1 <- x
call Q
add tmp, r0
t3 <- M[t1]
r3 <- t3
t4 <- M[t2]
r4 <- t4
ret
```

instr	def	use	in	out
t3 <- r3	t3	r3	r1, r2, r3, r4	r1, r2, r4, t3
M[t1] <- t3		t3	r1, r2, r4, t3	r1, r2, r4
t4 <- r4	t4	r4	r1, r2, r4	r1, r2, t4
M[t2] <- t4		t4	r1, r2, t4	r1, r2
x <- r1	x	r1	r1, r2	r2, x
y <- r2	y	r2	r2, x	x, y
r1 <- y	r1	y	x, y	r1, x
call Q	r0, r1, r2	r1	r1, x	r0, x
tmp <- r0	tmp	r0	r0, x	tmp, x
r1 <- x	r1	x	tmp, x	tmp, r1
call Q	r0, r1, r2	r1	tmp, r1	tmp, r0
add tmp, r0	r0	tmp, r0	tmp, r0	r0
t3 <- M[t1]	t3		r0	r0, t3
r3 <- t3	r3	t3	r0, t3	r0, r3
t4 <- M[t2]	t4		r0, r3	r0, r3, t4
r4 <- t4	r4	t4	r0, r3, t4	r0, r3, r4
ret			r0, r3, r4	r0, r3, r4

重新构造冲突图如下:



合并 t3, r3 (George)

合并 t4, r4 (George)

合并 y, r2 (George)

所有传送指令变为受抑制的, 可以消除

简化 tmp

简化 x

分配 r3 给 tmp, r4 给 x

染色成功, 重写指令并消除多余传送指令, 结果如下:

```
M[t1] <- r3
M[t2] <- r4
r4 <- r1
r1 <- r2
call Q
r3 <- r0
r1 <- r4
call Q
add r3, r0
r3 <- M[t1]
r4 <- M[t2]
ret
```

2018

Problem 4: Register allocation (40 points)

Suppose a compiler has compiled the following function **nfactor** into instructions on the right:

int nfactor (int n) { if (n <= 1) return 1; return n * nfactor(n - 1); }	nfactor: t1 <- r3 t2 <- r4 n <- r1 cmp n, 1 jle end r1 <- (n - 1) call nfactor r1 <- r1 * n jmp ret end: r1 <- 1 ret: r3 <- t1 r4 <- t2 return
---	---

Several things are worth noting in the instructions:

- There are **four** hardware registers in all.
- The compiler passes parameters using registers. (The first parameter goes to r1)
- r1, r2 are **caller-saved** while r3, r4 are **callee-saved**.
- The **return value** will be stored in r1.
- t1, t2, n are temporary registers

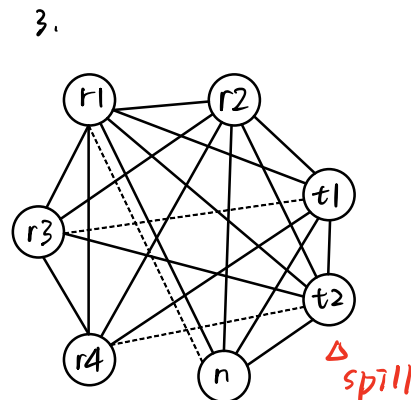
Please answer the following questions according to the instructions.

1. Draw a control flow graph instruction-by-instruction. (5')
2. Fill up the following def/use/in/out chart. (10')

Instr	def	use	in	out
t1 <- r3	t1	r3	r1, r3, r4	t1, r1, r4
t2 <- r4	t2	r4	t1, r1, r4	t1, t2, r1
n <- r1	n	r1	t1, t2, r1	n, t1, t2
cmp n, 1	-	n	n, t1, t2	n, t1, t2
jle end	-	-	n, t1, t2	n, t1, t2
r1 <- (n - 1)	r1	n	n, t1, t2	n, r1, t1, t2
call nfactor	r1, r2	r1	n, t1, t1, t2	n, r1, t1, t2
r1 <- r1 * n	r1	r1, n	n, r1, t1, t2	r1, t1, t2
jmp ret	-	-	r1, t1, t2	t1, t2
r1 <- 1	r1	-	t1, t2	r1, t1, t2
r3 <- t1	r3	t1	r1, t1, t2	r1, r3, t2
r4 <- t2	r4	t2	r1, r3, t2	r1, r3, r4
return	-	-	r1, r3, r4	r1, r3, r4

3. Draw the interference graph for the program. Please use dashed lines for move edges and solid line for real interference edges. (10')

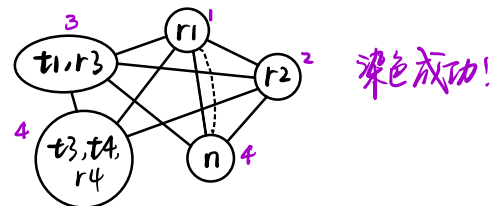
1.
t1 <- r3
t2 <- r4
n <- r1
cmp n, 1
jle end
r1 <- (n - 1)
call nfactor
r1 <- r1 * n
jmp ret
→ r1 <- 1
r3 <- t1
r4 <- t2
return



4. Adopt the graph-coloring algorithm to allocate registers for temporaries. (15')
 - a) If there exists any spilling:
 - i. rewrite the instructions and fill the chart below.

Instr	def	use	in	out
t1 <- r3	t1	r3	r1, r3, r4	t1, r1, r4
t3 <- r4	t3	r4	t1, r1, r4	t1, r1, t3
M[t2] <- t3	-	t3	t1, r1, t3	t1, r1
n <- r1	n	r1	t1, r1	t1, n
cmp n, 1	-	n	t1, n	t1, n
jle end	-	-	t1, n	t1, n
r1 <- (n - 1)	r1	n	t1, n	r1, t1, n
call nfactor	r1, r2	r1	r1, t1, n	r1, t1, n
r1 <- r1 * n	r1	r1, n	r1, t1, n	r1, t1
jmp ret	-	-	r1, t1	t1, t1
r1 <- 1	r1	-	t1	r1, t1
r3 <- t1	r3	t1	r1, t1	r1, r3
t4 <- M[t2]	t4	-	r1, r3	r1, r3, t4
r4 <- t4	r4	t4	r1, r3, t4	r1, r3, r4
return	-	-	r1, r3, r4	r1, r3, r4

- ii. Draw the new interference graph according to the chart.
 - b) Write down the final instructions after register allocation.
- NOTE:** Please make sure that you write the procedure clearly. Otherwise, you can get part of scores if you just provide the final instructions.



重写指令:

r4 → t3, t4, n
r3 → t1

M[t2] <- r4

r4 <- r1

cmp r4, 1

jle end

r1 <- (r4 - 1)

call nfactor

r1 <- r1 * r4

jmp ret

r1 <- 1

r4 <- M[t2]

return

2019

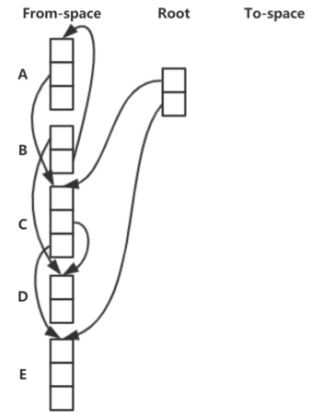
Problem 3: Garbage Collection (20 points)

1. Compare copying collection with mark & sweep collection, what's the **pros & cons** of **copying collection**?(4')

2. In the course, we have learnt that garbage collection help us recycle heap space occupied by non-live variables. How does the program reuse **space in stack frame(spills in pre-allocated stack) occupied by non-live variables**?(3')

3. When a garbage collection based on reachability analysis is about to happen, the program runtime usually scan stack frame and use reference variables in stack as roots. **If a reference variable is not live, will we still use it as a root? Why?**(3')

4. Use copying collection to finish the following datagram. You need to draw the memory state and pointer after **forwarding roots** and **scanning**. You can refer to **figure 13-4 in the Chinese textbook** or **figure 13.10. in the English textbook**. (10')



1.

优点：减少内存碎片化，操作简单复杂性低。

缺点：如果大部分变量存活时间很长，会导致内存拷贝过多，**overhead** 很大，而且复制回收只能用一半内存空间。

2.

对溢出变量染色，进行内存分配，这样可以把活跃范围不重叠的溢出变量分配到同一个内存地址处。

3.

会，活性分析是编译时的工作，而垃圾回收是运行时做的，它无法获取到变量的活跃信息。

4.

